CSCI/DSCI 575 Advanced Machine Learning
Project Report: Privacy-Preserving Record Linkage

Theodore (Ted) Cadieux

1. Problem Statement

Organizations across healthcare, financial services, and marketing routinely need to determine whether records in separate databases refer to the same real-world individual. This is the *record linkage* problem. A hospital network may need to reconcile patient records across providers; a bank may need to flag duplicate accounts created under slightly different spellings; a marketing data cooperative may need to match customer profiles held by two different companies.

The central difficulty is that high matching accuracy has historically required centralizing the data. Centralization exposes sensitive PII to a third party and creates legal and reputational risk under HIPAA, GDPR, and similar regulations. Organizations therefore want a system that links records accurately while reducing the amount of raw PII that leaves their own systems. The question this project investigates is: **how far can we push record linkage accuracy before the trust model we require becomes the bottleneck?**

2. Solution

This project implements and compares five record-linkage approaches on a common synthetic dataset, common similarity feature set, and common evaluation protocol. The five tiers form a deliberate progression so the gap between any two consecutive tiers isolates one specific cost:

Tier	Method	Trust model	F1
T1	Hand-tuned weighted scorecard (no ML)	Central party sees all raw data	0.9509
T2	Gaussian Naive Bayes	Central trainer sees features + labels	0.9996
T3	Centralized logistic regression	Central trainer sees features + labels	0.9985
T4	Federated logistic regression (FedAvg) + TTP	TTP computes features; clients train	0.9953
T5	FedAvg on Bloom-encoded text + salted hashes	No plaintext PII crosses parties	0.9990

- **T1 → T2** measures what learning buys over static weights.
- **T2 → T3** contrasts generative ($P(\text{features} \mid \text{class})$ + Bayes' rule) with discriminative ($P(\text{class} \mid \text{features})$ directly) modeling on the same features.

- **T3 → T4** measures the cost of federated optimization, holding the privacy footprint constant.
- **T4 → T5** measures the effect of swapping plaintext features for Bloom-filter encodings (text fields) and salted hashes (structured fields).

T5 is the first tier where the privacy claim genuinely holds. T1 through T4 still allow a trusted third party or central trainer to see raw records.

3. Demo

The project is implemented as six Jupyter notebooks that run in numerical order. Every result in the report can be reproduced by running `python Project/run_all.py` from the repo root, which executes the six notebooks in order. Each notebook is self-contained and documents its own approach.

All notebook paths below are relative to `Notebooks/`; output paths are relative to `Results/`.

1. `01_tier1_hand_tuned.ipynb`
   writes `tier1_metrics.csv` and `tier1_score_distribution.png`.
2. `02_tier2_naive_bayes.ipynb`
   writes `tier2_metrics.csv` and `tier2_score_distribution.png`.
3. `03_tier3_centralized_ml.ipynb`
   writes `tier3_metrics.csv` and `tier3_score_distribution.png`.
4. `04_tier4_federated_ttp.ipynb`
   writes `tier4_metrics.csv` and `tier4_score_distribution.png`.
5. `05_tier5_federated_bloom.ipynb`
   writes `tier5_metrics.csv` and `tier5_score_distribution.png`.
6. `06_results_comparison.ipynb`
   loads all five tiers outputs, produces the cross-tier metrics, the example-pairs, and the distribution plots.

Shared infrastructure lives in `utils.py`: paths, constants, plaintext similarity helpers, data loading, plaintext feature construction, metrics, and score-distribution plots.

Three fixed example pairs are re-used across every tier to follow how the same three records are predicted under each model. Per-tier example pair predictions are saved to `tier{n}_example_predictions.csv` and pivoted together in `06_results_comparison.ipynb`.

4. Assumptions, Constraints, and Implications

Assumptions

- Synthetic data with controlled corruption, not real-world PII. Organic noise (nicknames, schema mismatches, OCR errors) is not modeled.
- Candidate pairs are pre-built and labeled.
- Parties in T4 and T5 are honest: they follow the protocol but may inspect received weights. Active adversaries are not considered.
- T5's Bloom encoding obfuscates plaintext without providing a formal privacy guarantee (Kuzu et al. 2011 documents attacks).

Constraints

- T4's privacy footprint equals T3's. The TTP receives raw records before any federated training happens, so the federated step itself does not reduce exposure. T3 $\rightarrow$ T4 measures the training-approximation cost of FedAvg, not a privacy gain.
- T5 changes the trust model but adds no formal privacy guarantee. A production PPRL system would pair T5's encoding with DP-SGD during training, which was out of scope here.
- Single corruption level (30% per-field), fixed 1:9 class balance, uniform-random non-matches, and Bloom-filter defaults from Schnell et al. 2009.

Implications

- FedAvg alone is not a privacy mechanism. A TTP-based feature stage, shared labels, or naively averaged weights all preserve leakage pathways. Federated training has to be combined with a private feature stage and secure aggregation to change the trust model.
- Bloom-filter encoding is not a meaningful accuracy cost. T5 lands within 0.001 F1 of every centralized ML baseline. "Encoding must cost accuracy" is not a foregone conclusion; on richer-vocabulary tasks the effect may reverse.
- Errors across every tier concentrate on the recall side while precision stays at or above 0.999. For record linkage, this is the desired failure: a wrongly merged record generally costs more than a missed link.

5. How the Solution Was Built

Data

Synthetic records were generated with Faker (US names, street addresses, cities, states, ZIPs, DOBs, emails, phone numbers). 50,000 clean records make up Party A, and a corrupted copy makes up Party B. Each field in Party B has a 30% chance of being corrupted: text fields get one of `substitute`, `delete`, or `blank` applied; digit-bearing fields (ZIP, phone, DOB) get an adjacent-digit swap.

On top of the per-field typo corruption, two fields are dropped entirely at higher rates to more closely mirror real-world scenerios:

- Email entirely missing in 20% of records.
- Phone entirely missing in 25% of records.

The labeled pair file contains 50,000 true matches plus 450,000 random non-matches. Non-matches are uniformly sampled pairs where `rec_id_a` $\neq$ `rec_id_b`.

Feature engineering

Each candidate pair is represented by an 8-dimensional similarity vector. The structure of the vector is the same across all tiers; the way each entry is computed is where T5 diverges from T1 through T4.

Tiers 1–4 (plaintext features). Implemented as `build_feature_matrix()` in `utils.py`.

- Jaro-Winkler similarity on first name, last name, street address, date of birth, and email. Handles typos, transpositions, and partial edits.
- Normalized exact match on ZIP code, state, and phone. Formatting is stripped first (digits-only for ZIP and phone; uppercase and trim for state), then an equality check is applied.

Tier 5 (encoded features). Implemented inline in `05_tier5_federated_bloom.ipynb` as `build_bloom_feature_matrix()`.

- Dice coefficient on Bloom-filter encodings for the five text fields. Each string is encoded following Schnell et al. 2009 defaults.
- Equality on salted SHA-256 hashes for ZIP, state, and phone. Each normalized value is prefixed with a shared salt and hashed; parties compare hashes for equality, recovering the same 1.0/0.0 signal as plaintext equality without exchanging the raw values.

All eight features land in $[0, 1]$ in both variants. Missing or blank fields score 0.0. Both feature matrices use the same column order, so every downstream component works unchanged when T5 substitutes one for the other.

Models

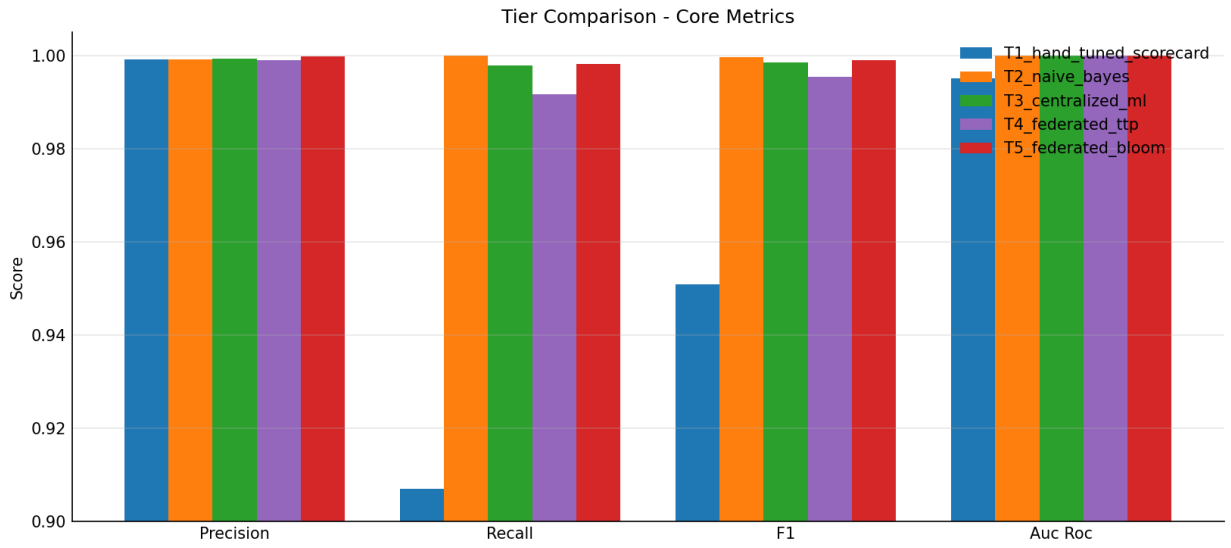
- **Tier 1 (hand-tuned scorecard).** A weighted sum of the eight per-field similarities with hand-picked weights (last name 0.25, DOB 0.20, first name 0.20, email 0.15, street 0.10, ZIP 0.05, phone 0.03, state 0.02; summing to 1.0) and a 0.70 threshold. No training involved and serves as the no-ML baseline.
- **Tier 2 (Gaussian Naive Bayes)** `sklearn.naive_bayes.GaussianNB` fits a Gaussian (μ, σ^2) per feature per class by maximum likelihood, plus the class prior $P(\text{match}) = 0.10$. Prediction applies Bayes' rule.
- **Tier 3 (Centralized LR).** `sklearn.LogisticRegression` with default L2 regularization, trained on 80% of the labeled pairs and evaluated on the held-out 20%. Classification threshold is 0.50 on the predicted probability. No scaling is applied since every feature is already in $[0, 1]$, which keeps the coefficients comparable to Tier 1's hand-tuned weights.
- **Tier 4 (Federated LR with TTP).** Same logistic regression model as T3 but trained via FedAvg + mini-batch SGD. The 80% training pool is split evenly across two simulated parties. In each of 100 communication rounds each party runs 2 epochs of mini-batch SGD, then the server averages the two returned weight vectors. Convergence ramps from F1 = 0.988 at round 20 to F1 = 0.9953 at round 100.
- **Tier 5 (federated LR with Bloom features).** Identical FedAvg training loop as T4 (same 2 parties, 100 rounds, 2 local epochs). The only difference is the input feature matrix: Dice-on-Bloom for text fields and salted-SHA-256 equality for structured fields. Convergence ramps from F1 = 0.997 at round 20 to F1 = 0.9990 at round 100.

6. Results

Aggregate metrics

All five tiers are evaluated on the same 100k held-out test set (same `random_state=42` split). T1 has no training so it still scores all 500k pairs, but only the 100k test subset is used for the reported metrics, making the tiers directly comparable.

Tier	Precision	Recall	F1	AUC-ROC
T1	0.9992	0.9071	0.9509	0.9950
T2	0.9992	1.0000	0.9996	1.0000
T3	0.9993	0.9978	0.9985	1.0000
T4	0.9990	0.9917	0.9953	1.0000
T5	0.9998	0.9982	0.9990	1.0000



Naive Bayes is the highest-F1 tier on this task, with the privacy-preserving encoded tier (T5) close behind and slightly above the centralized LR baseline.

- **T1** loses nearly 10 points of recall. 30% per-field corruption plus 20%/25% missingness on email and phone push many matches below the 0.70 threshold. Fixed weights can't redistribute to whatever fields survive per record. Precision stays near-perfect because random non-matches almost never clear 0.70.
- **T2 (Naive Bayes)** achieves perfect recall on the test set with precision around 0.999. The class-conditional Gaussians learn a sharp separation: per-feature means in the match class are 0.72–1.00 across the board, while non-match means sit at 0.00–0.71.
- **T3 (centralized LR)** lands marginally below T2. The model re-weights features to handle missingness: the `email` coefficient collapses (unreliable, 20% missing) and `date_of_birth` dominates at coefficient 14.7. T2 vs T3 illustrates the generative-vs-discriminative tradeoff at small scale; on this feature set, generative wins narrowly.
- **T4 (FedAvg with TTP)** drops about 0.003 F1 from T3. FedAvg costs a small amount of recall on borderline pairs. The privacy footprint is unchanged from T3 (the TTP still sees raw records), so this gap is the pure training-approximation cost.
- **T5 (FedAvg + Bloom)** edges above T3 and well above T4. Dice-on-Bloom is less generous than Jaro-Winkler to unrelated pairs while still scoring corrupted and related strings highly, so the encoded feature distribution turns out to be marginally better for FedAvg than the plaintext one.

Example pair comparison

Three fixed pairs are reused across all tiers. All tier notebooks save their predictions for these pairs; the comparison notebook joins them into a single table (`comparison_example_pairs.csv`):

Pair	Match	T1	T2 (NB)	T3 (LR)	T4 (FedAvg)	T5 (Bloom)
Clear match (rec 11841)	True	0.993	1.000	1.000	1.000	1.000
Missed match – FN (rec 10516)	True	**0.700**	1.000	**0.967**	**0.534**	**0.832**
Non-match (47135 / 39460)	False	0.447	0.000	0.000	0.000	0.000

The missed-match row is the most interesting cell. Record 10516 (detailed below) has its first name blanked, a one-character last-name typo (`Williams` becomes `Wlliams`), a ZIP with two adjacent digits transposed (`10495` vs `10945`), and a missing phone; street address, DOB, state, and email all match. T1's weighted score (0.6999) lands just below the 0.70 threshold and the pair is classified as a non-match. All four ML tiers catch it, but with different margins: T3 is confident at 0.967; T5 is confident at 0.832; T4 only barely clears the 0.50 threshold (0.534). The spread mirrors the aggregate ordering.

The two records side by side (corrupted values in **bold**, blanked values in *italics*):

Field	Party A	Party B
first_name	Amanda	***(blank)***
last_name	Williams	**Wlliams**
street_address	11762 Amanda Crossroad Suite 276	11762 Amanda Crossroad Suite 276
state	MO	MO
zip_code	10495	**10945**
date_of_birth	1990-04-09	1990-04-09
email	danacook@example.com	danacook@example.com
phone	739.785.8166	***(blank)***

What the ML models learned

T2 (Gaussian Naive Bayes). Class priors $P(\text{non-match}) = 0.90$, $P(\text{match}) = 0.10$. Per-feature class-conditional means cleanly separate the two populations:

Feature	μ (non-match)	μ (match)
`first_name`	0.345	0.886
`last_name`	0.327	0.884
`street_address`	0.471	0.894
`date_of_birth`	0.708	0.993
`email`	0.466	0.715
`zip_code`	0.000	0.731
`state`	0.017	1.000
`phone`	0.000	0.596

The match-class mean for `state` is 1.000 and for `zip_code` is 0.731 (with non-match means near zero). These are the cleanest signals because they're encoded as exact-match. `email` has the smallest separation (0.466 vs 0.715), reflecting its 20% missingness.

T3 (centralized LR). Largest coefficient `date_of_birth` (14.68), then `zip_code` (10.01), `phone` (9.55), `state` (9.21). `email` is the lowest at 2.15. The model learned to discount a field that's missing 20% of the time.

T4 (FedAvg, plaintext). Flatter coefficients: `state` (5.74) on top, then `zip_code` (5.12) and `phone` (4.49). `date_of_birth` is essentially zero (0.08); FedAvg struggled to drive it up within the round budget. Intercept −11.92. Magnitudes are roughly a third of T3's, which is why T4 is less confident on borderline pairs.

T5 (FedAvg, Bloom). Lands close to T4 on top features (`state` 4.89, `zip_code` 4.03) but spreads weight more evenly across `first_name`, `last_name`, and `phone` (3.45–3.74). `date_of_birth` gets a meaningful positive coefficient (2.49) instead of T4's near-zero. DOB agreement survives the salted-hash encoding losslessly, so the model has clean binary signal on it; FedAvg in T5 incorporated that signal where T4 couldn't.

Score distributions

Each tier produces a histogram of its output (weighted score for T1; predicted probability for T2 through T5), all built from the same 100k test set. T1's distribution shows $\sim$ 930 corrupted matches with scores in the 0.25–0.70 range; these are the false negatives. T2 (NB) and T3 (LR) are both cleanly bimodal (matches near 1, non-matches near 0, almost nothing in between). T4 (FedAvg, plaintext) leaves a small secondary cluster of matches at probability 0.3–0.6, the borderline cases its flatter coefficients struggle with. T5 recovers most of that cluster, bringing the match distribution closer to T2/T3's bimodal shape. See `Results/tier{n}_score_distribution.png`.

7. Summary

This project built and evaluated a five-tier framework for record linkage on synthetic data with realistic corruption (30% per-field typo rate, plus 20% missing email and 25% missing phone on Party B) and 50k matches / 450k random non-matches at a 1:9 ratio. All five tiers are evaluated on the same 100k held-out test set so numbers compare directly.

The central quantitative findings:

- **Naive Bayes (T2) wins on F1 (**0.9996**)** with perfect recall on the test set. The generative model, class-conditional Gaussians fit by MLE plus a class prior, fits this task well.
- **Centralized LR (T3) reaches F1** = 0.9985**.** Adapts to missingness by re-weighting features. `email` collapses (unreliable), `date_of_birth` dominates at coefficient 14.7. T2 vs T3 contrasts generative and discriminative modeling at small scale; generative narrowly wins.
- **Hand-tuned T1 (F1** = 0.9509**)** pays a large penalty for static weights under this corruption level. Recall falls to 0.907 ($\sim$ 930 missed matches on 10k positive test pairs). The T1 $\rightarrow$ T2/T3 gap of about 0.05 F1 is the cost of not learning at all.

- **FedAvg T4 (F1 = 0.9953)** trails T3 by about 0.003 F1. This is the pure training-approximation cost of FedAvg with the privacy footprint held constant. T4 is not a privacy improvement over T3.
- **FedAvg + Bloom T5 (F1 = 0.9990)** is the second-highest-F1 tier (slightly above T3 and well above T4). It is also the only tier where no plaintext PII crosses between parties.

The most important qualitative finding is that T5 genuinely earns the "privacy-preserving" label and lands within 0.001 F1 of every centralized baseline. T1 through T4 all assume a central party or TTP sees raw records; T5 replaces plaintext with encodings before any cross-party exchange. On this task the encoded feature distribution turns out to be slightly cleaner than the plaintext one. Dice-on-bigrams suppresses random-pair similarity without sacrificing signal on corrupted-but-real matches, and salted-hash equality on DOB/state/ZIP gives FedAvg a clean signal that plaintext Jaro-Winkler doesn't.

The accuracy-privacy decomposition this framework provides is practically useful: the hand-tuned baseline pays about 0.05 F1 (T1 $\rightarrow$ T2/T3); federated training costs about 0.003 F1 (T3 $\rightarrow$ T4); encoded features gain about 0.004 F1 (T4 $\rightarrow$ T5); and the full privacy-preserving pipeline (T5) lands within 0.001 F1 of the best centralized result. On this task, the privacy tradeoff story largely inverts: privacy-preserving features and federated training are essentially free.

Known issues and limitations include synthetic data with controlled corruption, a simulated two-party federated setup, the absence of a formal differential-privacy guarantee, and the lack of a formal privacy guarantee for the Bloom-filter encoding step. The natural next steps are to add DP-SGD during local training for a formal (ε, δ)-DP bound, to replace blocking with a Private Set Intersection protocol, and to validate on a real-world dataset with organic noise such as FEBRL or Ohio electronic health records.

References

Abadi, M., Chu, A., Goodfellow, I., McMahan, H.B., Mironov, I., Talwar, K., & Zhang, L. (2016). Deep learning with differential privacy. *ACM CCS 2016*.

Dwork, C., & Roth, A. (2014). The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science, 9*(3–4), 211–407.

Kairouz, P., McMahan, H.B., et al. (2021). Advances and open problems in federated learning. *Foundations and Trends in Machine Learning, 14*(1–2), 1–210.

Kirsch, A., & Mitzenmacher, M. (2008). Less hashing, same performance: Building a better Bloom filter. *Random Structures & Algorithms, 33*(2), 187–218.

Kuzu, M., Kantarcioglu, M., Durham, E., & Malin, B. (2011). A constraint satisfaction cryptanalysis of Bloom filters in private record linkage. *PETS 2011*.

McMahan, H.B., Moore, E., Ramage, D., Hampson, S., & Aguera y Arcas, B. (2017).

Communication-efficient learning of deep networks from decentralized data. *AISTATS 2017*.

Schnell, R., Bachteler, T., & Reiher, J. (2009). Privacy-preserving record linkage using Bloom filters. *BMC Medical Informatics and Decision Making, 9*(1), 41.

Winkler, W.E. (1990). String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage. *Proceedings of the Section on Survey Research Methods*, American Statistical Association, 354–359.